

BM25 in This Repository — How Lexical Retrieval Actually Scores

Reference for the scoring function behind S5 retrieval, grounded in the code that runs it: `src/index.py` (CatalogIndex), `src/retrieval.py` (retrieve), `src/text.py` (terms). Companion docs: `IMPLEMENTATION.md` §S1/§S5, `docs/team/category_tail_match.md` (the worked document-frequency failure), `docs/team/stress_harness.md` (paraphrase recall numbers), `docs/team/signal_descriptions.md` §3–§4 (why the reranker inherits BM25 only as a *ranking*, never as a *filter*).

The one-sentence version is in the last section. Everything before it is the mechanism.

0. Where BM25 sits

```
conversation turns
  |
  v
src/text.terms()      lowercase, dedupe, drop stopwords + boilerplate, cap 60
  |
  v
CatalogIndex.search_terms() -> '"tok1" OR "tok2" OR ... OR "tokN"'
  |
  v
SQLite FTS5:  SELECT parent_asin, bm25(products, <weights>) AS rank
              FROM products WHERE products MATCH ? ORDER BY rank LIMIT ?
  |
  v
_match():  return [(asin, -float(rank)), ...]      # negate: bigger = better
  |
  v
retrieve(): 3 routes (anchor / terms / focused) fused by reciprocal rank
  |
  v
pool of <= 300 candidates -> S6 rerank
```

BM25 is the only thing that reads the full 50,000-product catalog per turn. The reranker never calls `bm25()` again; it inherits the fused *rank positions* and rescoring is done on verbatim span containment (`docs/team/signal_descriptions.md`).

1. The BM25 formula

BM25 scores one document d against a bag-of-words query Q as a sum of independent per-term contributions:

$$\text{BM25}(d, Q) = \sum_{t \text{ in } Q} \frac{\text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1} \cdot \frac{1}{(1 - b + b \cdot |d| / \text{avgdl})}}{1}$$

Every symbol, defined before it is used:

symbol	meaning
t	one query token (after <code>terms()</code> normalisation)
$f(t, d)$	term frequency: how many times t occurs in document d
$\text{IDF}(t)$	inverse document frequency of t across the whole catalog (below)
$ d $	length of document d (token count)
avgdl	mean document length over all N catalog rows
N	number of catalog rows (~50,000 here)

n _t	number of catalog rows that contain t at least once ("document frequency")
k ₁	term-frequency saturation constant — 1.2 in FTS5
b	length-normalisation strength, 0..1 — 0.75 in FTS5

1a. IDF — rare terms score higher

$$\text{IDF}(t) = \ln((N - n_t + 0.5) / (n_t + 0.5))$$

A term in one product out of 50,000 is enormously informative; a term in 30,000 of them is nearly worthless. Worked, at $N = 50,000$:

term (real catalog counts)	n _t	share	IDF
a genuinely rare token (e.g. a brand coinage)	5	0.01%	9.11
all other heathers (as a phrase span)	513	1.0%	4.57
100 cotton	3,774	7.5%	2.51
closure (before boilerplate drop)	~19,300	38.6%	0.45
a token in half the catalog	25,000	50%	~ 0
a token in most of the catalog	40,000	80%	negative

(n_t values for the phrase spans are from docs/team/category_tail_match.md §2; closure 38.6% is from the comment in src/text.py.) FTS5 **floors** a non-positive IDF at a tiny epsilon, so a term in more than ~half the catalog contributes essentially nothing rather than subtracting. This is why src/text.drop_boilerplate removing closure, imported, wash, machine etc. changes the public score by *nothing* (IMPLEMENTATION.md §S1, "Measured effect: none"): those tokens were already near-zero-weight inside BM25. The drop-list is for query cost and audit, not points.

1b. TF saturation — the 5th occurrence barely matters

Hold $|d| = \text{avgdl}$ so the bracket's denominator factor is 1. The TF term $f \cdot (k_1 + 1) / (f + k_1)$ with $k_1 = 1.2$:

f(t, d)	TF term	gain over previous
1	1.000	—
2	1.375	+0.375
3	1.571	+0.196
5	1.774	+0.087 (per step)
10	1.964	+0.019 (per step)
∞	2.200 (= k ₁ + 1)	asymptote

A product that repeats "leather" five times is **not** five times more relevant than one that says it once — it is about 1.77x on that term. The curve is concave and bounded by $k_1 + 1 = 2.2$. Small k_1 = fast saturation; FTS5's 1.2 saturates quickly.

1c. Length normalisation — short documents score higher

The denominator factor $1 - b + b \cdot |d|/\text{avgdl}$ with $b = 0.75$ is $0.25 + 0.75 \cdot (|d|/\text{avgdl})$. For a single occurrence ($f = 1$, $k_1 = 1.2$), the TF term becomes $2.2 / (1 + 1.2 \cdot \text{factor})$:

document length	factor	TF term at f=1
-----------------	--------	----------------

half average (thin listing)	0.625	1.257
average	1.000	1.000
2x average	1.750	0.710
3x average (rich, multi-bullet listing)	2.500	0.550

Same single mention of the query term, **2.3x** the score for the thin listing over the 3x-length one. $b = 0$ would switch normalisation off entirely; $b = 1$ would divide fully by relative length. 0.75 is a strong-ish penalty. Hold onto this row — §5a is the failure it causes in this repo.

1d. The sign convention

FTS5's `bm25()` returns $-1 \times \text{BM25}(d, Q)$, so a better match is a *more negative* number and `ORDER BY rank` (ascending) puts the best row first. `src/index.py:_match()` immediately flips it back:

```
# bm25() returns increasingly negative values for better matches.
return [(str(asin), -float(rank)) for asin, rank in rows]
```

so everything downstream sees a normal "bigger is better" score.

2. FTS5 does BM25F, not textbook BM25

`bm25(products, w0, w1, ...)` is **fielded** BM25 (BM25F). The products virtual table has seven columns (`src/index.py:FTS_COLUMNS`), and the weight tuple is positional over them:

index	column	DEFAULT_WEIGHTS	what it holds
0	parent_asin	0.0	the id — UNINDEXED, never contributes
1	title	8.0	product title
2	categories	5.0	flattened category path
3	features	6.0	bullet list, flattened
4	details	6.0	the details dict, flattened to key value key value
5	store	0.5	brand / storefront name
6	description	0.25	long marketing prose

`DEFAULT_WEIGHTS = (0.0, 8.0, 5.0, 6.0, 6.0, 0.5, 0.25)` (`src/index.py:28`).

What "column" means here. At index time (`CatalogIndex._build()`) each catalog record is flattened field-by-field and inserted as one FTS5 row with seven cells. `flatten()` (`src/text.py`) renders lists as space-joined items and dicts as "key value key value". `parent_asin` is declared UNINDEXED, so it is stored but not tokenised — combined with weight 0.0 it can never match or score.

How the columns combine. FTS5 computes one BM25F score per row by folding the per-column term frequencies and lengths together with the weights (this is `fts5_aux.c`'s `bm25()` implementation):

```
f(t, d) = SUM_c w_c · tf(t, d_c)      # weighted term frequency
|d|      = SUM_c w_c · |d_c|           # weighted document length
avgdl    = SUM_c w_c · (SUM_rows |d_c| / N)  # weighted average length
```

then feeds those into the single formula from §1. So a hit is not scored per column and then summed — the columns are merged into one *weighted* pseudo-document and scored once, with $k1 = 1.2$, $b = 0.75$.

Consequence of the shipped weights. A term appearing once in `title` contributes as if it appeared 8 times; the same term in `description` contributes as if 0.25 times. A `title` hit is therefore worth **32x** a `description` hit ($8.0 /$

0.25), and worth 16x a store hit. categories, features and details sit in the strong middle band (5–6). The intent is exactly what `src/index.py`'s comment says: "favour title and categories over long marketing description text", so that a product whose *title* is about belts outranks one that merely mentions belts in a paragraph.

These weights are inherited, not tuned. `IMPLEMENTATION.md` §S1 flags them as "probably the cheapest untapped gain in the repo" — the sweep harness exists, the parameter is one tuple, and nobody has verified the baseline's guess.

3. Turning a conversation into a query

3a. `terms()` (`src/text.py`)

- 1 Tokenise on `[a-z0-9]+` (the FTS5 tokenizer is `unicode61 remove_diacritics 2`, so both sides lowercase and strip accents).
- 2 Drop tokens shorter than 2 characters.
- 3 Drop STOPWORDS — conversational filler the simulator emits: `i, looking, for, want, prefer, requirement, the, ...`
- 4 When `drop_boilerplate=True` (the `search_terms` default) also drop BOILERPLATE = learned SCAFFOLDING (Amazon structural metadata: month names, dimensions, manufacturer, ...) plus hand-kept CARE_PHRASES (`imported, closure, machine, wash, care, made, usa, ...`).
- 5 Keep first-seen order, **deduplicate** (so a token said twice is queried once).
- 6 Truncate to `MAX_QUERY_TERMS = 60` (never binds in practice — 0 of 200 public sessions come close).

3b. `search_terms()` (`src/index.py`)

```
tokens = terms(text, drop_boilerplate=drop_boilerplate)[:MAX_QUERY_TERMS]
return self._match(" OR ".join(f'"{token}"' for token in tokens), limit, weights)
```

The MATCH string is a **pure disjunction**: `"a" OR "b" OR "c"`. A row matches if it contains **any** one query token. Its score is the sum of the BM25F contributions of the tokens it *does* contain; missing tokens contribute nothing (they are not a penalty). Each token is wrapped in double quotes only to make it a literal FTS5 string, not to make a phrase — these are all single tokens.

There are no phrases at retrieval time. Verbatim / adjacency matching ("`stainless steel band` as a unit") is deliberately *not* a retrieval route: as an FTS5 phrase query it recalls the target in only 47 of 80 sampled sessions, so fusing it here injects more noise than signal. It lives in the reranker instead, as a rescoring signal over a pool the OR-query already filled (`src/retrieval.py` docstring; `IMPLEMENTATION.md` §S6).

3c. *Worked example*

Opening (turn 1): "I'm looking for Belts. A key requirement is: 100% Leather." Reply (turn 2): "For that, what matters is: Buckle closure; Machine wash cold."

`terms()` over the accumulated `full_text()`:

- tokenise -> `i m looking for belts a key requirement is 100 leather for that what matters is buckle closure machine wash cold`
- drop `<2` -> removes `i, m, a`
- drop stopwords -> removes `looking, for, key, requirement, is, that, what, matters`
- drop boilerplate -> removes `closure, machine, wash`
- dedupe, keep order -> `["belts", "100", "leather", "buckle", "cold"]`

FTS5 MATCH string actually issued by the **terms route**:

"belts" OR "100" OR "leather" OR "buckle" OR "cold"

The **anchor route** runs `search_terms(state.opening)` — turn 1 only — so it issues "belts" OR "100" OR "leather". The **focused route** issues nothing until an override fires (`state.override_turn` is not `None`), then runs over the post-override turns.

3d. The three routes and how they fuse (`src/retrieval.py`)

route	text queried	when	RRF weight
anchor	<code>state.opening</code>	always (turn 1 exists)	0.6
terms	<code>state.full_text()</code> (whole conversation, declines excluded)	always	1.0
focused	<code>state.focused_text()</code> (post-override turns)	only after an override	0.8

Each route returns up to `pool_size = 300` ranked rows. They are combined by **reciprocal-rank fusion**, not score addition:

```
RRF_K = 60.0
sink[asin] += weight / (RRF_K + position + 1)    # position is 0-based
```

A row at position 0 in the terms route contributes $1.0 / 61$; at position 9, $1.0 / 70$. RRF is used because the three routes' raw BM25F scores are on incomparable scales (different query strings, different lengths) and rank fusion needs no calibration — and a route that returns nothing simply contributes nothing. The fused pool is the top 300 by summed RRF score.

4. Why BM25 alone lands the target in the 300-pool ~100% of the time on this task

This is the central fact about retrieval in this repo, and it is an artifact of the **evaluator**, not a triumph of the scorer.

4a. The verbatim-disclosure artifact

`evaluator/local_evaluator.py:intent_card()` builds the simulated customer's constraints by **scraping the target product's own text**:

- `target_category` = the target's cleaned title (truncated to 180 chars).
- `hard_constraints / soft_preferences` = the first flattened items of the target's features and details, plus a material from `MATERIAL_RE.search(corpus)` inserted at position 0 and a colour from `COLOR_RE.search(corpus)` at position 1 — where `corpus = searchable_text(product)` is the target's own title + features + details + description + categories + store.
- The opening line is `f"I'm looking for {coarse_category}. A key requirement is: {constraint}."`, and `coarse_category()` is the **last two levels of the target's category path**.

`customer_reply()` then discloses further `hard_constraints / soft_preferences` items — still verbatim card text — when asked.

Every non-stopword token the customer ever utters is, by construction, present in the target document. The material, the colour, the category tail, every feature bullet fragment: all copied from the target's title / features / details / categories. So in the OR-query:

- the target has $f(t, d) > 0$ for **essentially every query token**;
- most of those tokens land in the high-weight columns (title 8.0, categories 5.0, features 6.0, details 6.0) because that is where the card scraped them from;

- an impostor typically matches a *subset* of the tokens.

The target therefore accumulates BM25F contributions on the whole query and ranks near the top of a disjunctive search. IMPLEMENTATION.md §S5 measured it on 80 sampled fully-disclosed sessions: **target in the pool 80/80, median position 1**, top-10 in 69/80 (the 11-session gap is *ordering*, handed to S6).

4b. What makes 300 a generous cutoff

- `pool_size = 300` out of ~50,000 rows is the **top 0.6%**.
- The query only **grows** across turns (state accumulates; declines are the only thing excluded), so a token that recalls the target on turn 2 keeps recalling it on turn 5.
- **Three routes**, RRF-fused: a target that one route ranks past 300 is still caught if another route ranks it inside 300. The anchor route (opening only) and the terms route (everything) fail on different sessions.

4c. The numbers

- Public set (200 sessions): **Hit@10 = 1.000** — the target is not just in the pool, it is in the *shown top 10*, every session (docs/team/category_tail_match.md §5).
- `future_steps.md` §1: "*BM25 recall is already ~100%*", which is why the dense (embedding) retrieval route is off by default and "never helped".
- Hard set (96 adversarial sessions): Hit@10 ~0.90. `tools/observe.py` on the baseline splits the 10 misses as **6** `never_retrieved` (target never in the pool) and **4** `ranked_out` (in the pool every turn, never surfaced — a *ranking loss*). The `never_retrieved` cases are the genuinely degenerate cards — e.g. `budget_only_signal` sessions whose only disclosed constraint is "budget around \$—" with no number and no catalog token to match (docs/team/signal_descriptions.md, "Not a signal: price / budget").

5. Where BM25 is weak — the failure modes this repo has hit

5a. Length normalisation rewards thin listings (the tie-break regime)

From §1c: for identical query-term coverage, the **shorter** document scores higher on the `retrieval_score` term. `src/rerank.py`'s module `docstring` records the consequence directly:

dissecting every near-miss session showed all lexical signals exactly tied 33/33 between the target and the impostor above it, with the tie broken by the retrieval score (wrong 33/33 — BM25 length normalization favours thin listings) while popularity pointed at the target 31/33.

The target is a **real purchase** — a reviewed, well-documented product with a long features / description, i.e. a *long* document, i.e. `|d|/avgdl > 1`, i.e. penalised by `b = 0.75`. A skeletal impostor listing with the same query tokens and almost no other text gets `|d|/avgdl < 1` and wins the normalised retrieval score. Because span coverage, facet agreement and category agreement all **saturate** inside a homogeneous cluster (every candidate matches the same disclosed boilerplate), that corrupted retrieval score was the *only* tie-break left — and it was pointing at the impostor.

The fix was not to BM25 but around it: `popularity_weight` was raised **0.02 -> 0.4** (`src/rerank.py:RerankConfig`), so the "is this a real, reviewed product" prior can overcome the length-normalisation bias in exactly that regime. Measured: dev 0.9268 -> 0.9418, public 0.9199 -> 0.9305, hard 0.7981 -> 0.8020, public 200/200 kept. The full anatomy is docs/team/category_tail_match.md and `rerank_signals.md` §10.

5b. Pure lexical — no synonymy, no morphology

FTS5's tokenizer does case-folding and diacritic stripping and nothing else: no stemming, no lemmatisation, no synonym expansion. `leather` and `cowhide` share zero tokens. `waterproof` and `water-repellent` tokenise to

{waterproof} vs {water, repellent} — no overlap on the head token. So the moment the customer **paraphrases** instead of quoting catalog text, the OR-query loses the tokens it was relying on.

docs/team/stress_harness.md quantifies it on the public set:

customer	Hit@10	token coverage
official (verbatim disclosure)	1.000	0.792
paraphrase:heavy (synonym substitution)	0.970	0.660
paraphrase:heavy + browse-gated	0.930	0.573

Per-scenario under heavy + browse-gated, the retrieval gap is **track-asymmetric**: gated + paraphrased *browsers* hit never_retrieved ~**10 / 80**, while *buyers* (who front-load a hard constraint in the opening) stay at **1 / 80**. Token coverage falling to ~0.57 is the mechanism: a vague, reworded browser gives the bag-of-words query one or two non-verbatim terms and it "has nothing to bite on". This is the standing argument for a dense / query-expansion route on the browsing track (stress_harness.md "Implications", point 2).

5c. OR-query with common terms — the target sits deep even when retrieved

If **every** disclosed constraint is a high-document-frequency phrase, IDF is low for all of them, the score spread across candidates collapses, and the target — still in the pool — sits at BM25 rank 200–300 behind hundreds of near-identical listings. docs/team/category_tail_match.md §2 is the worked case: the customer disclosed the Amazon printed-apparel fabric block, and although each span looked rare individually —

span	products containing it	share of 50,000
solid colors	708	1.4%
100 cotton	3,774	7.5%
heather grey	612	1.2%
90 cotton	649	1.3%
10 polyester	634	1.3%
all other heathers	513	1.0%
50 cotton	719	1.4%
50 polyester	744	1.5%

— **468 catalog products contain all eight**, and the pool (built from those very terms) concentrated them. The target fell from pool rank 38 to **273**. Products that repeat the fabric block across many colourway bullets accumulate more weighted TF than the target, which states it once — and TF saturation (§1b) only partly blunts that.

tools/stress_harness.py --targets generic isolates this: targets whose spans are *all* high-frequency (≥ 400 products each). Even fully cooperative, their MRR is 0.636 vs 0.868 overall; with paraphrase:heavy + browse-gated it collapses to 0.348. This is also why src/rerank.py sets depth = 300 (rescore the *whole* pool, not a prefix): "~12% of cluster-target sessions had the target in the pool but past rank 200", where the span signal never got to run.

6. Mental model

BM25 = "how many of the words you said show up in this product, weighted by how rare each word is across the catalog, how many times it appears here (with heavy diminishing returns after the first two or three), boosted if the word landed in the title or category rather than deep in the marketing blurb, and docked if this product's text is long." It has no idea that "leather" and "cowhide" mean the same thing, no idea that "belt" and "belts" are the same word beyond what plain lowercasing gives it, and it treats your turns as one unordered bag with

no phrases. On this task it recalls the right product almost every time — but only because the simulated customer reads its constraints straight off that product's own page.

Take the verbatim quoting away and two things degrade, unevenly. Across the whole set it is **ranking** that suffers first: the target usually stays in the pool (Hit@10 1.000 -> 0.970 at paraphrase:heavy) but the reranker's verbatim-span signal collapses, so it drops from rank 1 to rank 3–5 and MRR falls 0.87 -> 0.75. **Recall** breaks only where disclosure is *also* sparse — the gated + paraphrased browsing track, where never_retrieved climbs to ~10/80. So the first fix is a fuzzier (token-overlap) span match in the reranker; a dense retrieval route is the second, narrower fix for the browsing tail.